

Expt. No. : 12

Date :

USAGE OF EVAL() AND QUERY()

AIM

To write a Python program to demonstrate the use of eval() and query() methods using the Pandas library.

SOFTWARE REQUIRED

Python 3.14

THEORY

Pandas is a powerful data manipulation and analysis library in Python. It provides flexible data structures such as DataFrames and Series, allowing for efficient handling and analysis of structured data. Among its many features, Pandas offers methods like eval() and query() for performing operations on DataFrames in a more readable and expressive manner.

The eval() Method

The eval() method in Pandas allows for the evaluation of expressions involving DataFrame columns. It enables users to perform arithmetic and logical operations using a string expression, making the code cleaner and often faster by avoiding the need for intermediate variables.

Key Features of eval():

- **Expression Evaluation:** Evaluate expressions involving multiple columns in a DataFrame.
- **Performance:** Can be faster than using standard Python operations because it optimizes operations under the hood.
- **Readable Syntax:** Provides a more readable syntax for mathematical operations compared to standard DataFrame operations.

Syntax:

DataFrame.eval(expr, inplace=False, **kwargs)

- **expr:** The expression to evaluate.
- **inplace:** If True, updates the DataFrame in place; if False (default), returns a new DataFrame.
- **kwargs:** Additional keyword arguments that can be passed to the underlying eval method.

The query() Method

The query() method allows users to filter a DataFrame using a query expression in a more intuitive way. It provides a way to select rows based on column values, making it easier to perform complex filtering operations.

Key Features of query():

- **Filtering:** Filter rows of a DataFrame based on a query expression.

- **Readability:** Enhances the readability of filtering conditions compared to traditional indexing methods.
- **Support for Logical Operators:** Supports logical operators like and, or, and not, making it easy to combine multiple conditions.

Syntax:

DataFrame.query(expr, inplace=False, **kwargs)

- **expr:** The query expression to evaluate.
- **inplace:** If True, updates the DataFrame in place; if False (default), returns a new DataFrame.
- **kwargs:** Additional keyword arguments for passing additional variables.

PROBLEM

1. Build a Python program using Pandas to use eval() to calculate the total working hours from regular and overtime hours, and query() to display employees whose total working hours exceed a specified limit.

DATASET

```
data = {
    "ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Name": ["Arun", "Bala", "Charan", "Divya", "Ezhil", "Farah", "Gokul", "Hari"],
    "Marks": [85, 72, 91, 65, 78, 88, 95, 70],
    "Attendance": [92, 85, 96, 72, 88, 94, 98, 80],
    "Quantity": [5, 8, 3, 10, 6, 4, 7, 9],
    "Unit_Price": [200, 150, 500, 100, 250, 400, 300, 180],
    "Basic_Salary": [25000, 28000, 32000, 22000, 30000, 35000, 40000, 27000],
    "Allowance": [5000, 6000, 8000, 4000, 7000, 9000, 10000, 5500],
    "Regular_Hours": [8, 8, 9, 7, 8, 9, 8, 7],
    "Overtime_Hours": [2, 1, 3, 2, 1, 2, 4, 1],
    "Discount_Percent": [10, 5, 15, 10, 8, 12, 5, 10]
}
```

2. Build a Python program using Pandas to use eval() to calculate the power-factor-related useful power from apparent power and power factor, and use query() to identify equipment having a power factor below a specified limit.

DATASET

```
data = {
    "Equipment_ID": ["E001", "E002", "E003", "E004", "E005", "E006", "E007", "E008", "E009", "E010"],
    "Equipment": ["Transformer", "Motor", "Generator", "Transformer", "Motor", "Solar Panel", "Generator", "Motor", "Transformer", "Solar Panel"],
    "Voltage_V": [230, 415, 400, 230, 415, 240, 400, 415, 230, 240],
    "Current_A": [12, 15, 18, 10, 20, 8, 22, 17, 14, 9],
    "Power_kW": [2.76, 5.50, 7.20, 2.30, 7.10, 1.92, 8.80, 6.80, 3.22, 2.16],
    "Efficiency_Percent": [94, 91, 89, 95, 90, 87, 92, 91, 96, 88],
    "Operating_Hours": [8, 10, 12, 7, 9, 6, 11, 10, 8, 7],
    "Power_Factor": [0.92, 0.88, 0.85, 0.94, 0.87, 0.96, 0.84, 0.89, 0.93, 0.95],
    "Temperature_C": [35, 42, 48, 33, 45, 38, 51, 44, 36, 40]
}
```

3. The `eval()` expression is a very powerful built-in function of Python. It helps in evaluating an expression. The expression can be a Python statement, or a code object. Here, `eval()` can also be used to work with Python keywords or defined functions and variables. These would normally be stored as strings.

Task: You are given an expression in a line. Read that line as a string variable, such as `var`, and print the result using `eval(var)`.

ALGORITHM

STEP 1: Start the program.

STEP 2: Import pandas as `pd`

STEP 3: Get the data

STEP 4: Write the data in dataframe

STEP 5: Use `eval ()` to calculate a new column based on existing columns

STEP 6: Use `query()` to filter rows based on a condition

STEP 7: Print the dataframe after using `eval()` and `query()`

STEP 8: Terminate the program.

PROGRAM CODING

```

import pandas as pd
n = int(input("Enter number of employees: "))
data = []
for i in range(n):
    name = input("Enter employee name: ")
    regular = float(input("Enter regular hours: "))
    overtime = float(input("Enter overtime hours: "))
    data.append([name, regular, overtime])
df = pd.DataFrame(
    data,
    columns=["Name", "Regular_Hours", "Overtime_Hours"]
)
df["Total_Hours"] = df.eval("Regular_Hours + Overtime_Hours")
limit = float(input("Enter hour limit: "))
result = df.query("Total_Hours > @limit")
print("\nEmployees:")
print(result)

```

OUTCOMES OF PROGRAM CODING

```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Experiment\Experiment12.py =====
Enter number of employees: 3
Enter employee name: Arun
Enter regular hours: 8
Enter overtime hours: 2
Enter employee name: Bala
Enter regular hours: 7
Enter overtime hours: 1
Enter employee name: Charan
Enter regular hours: 9
Enter overtime hours: 3
Enter hour limit: 9

Employees:
   Name Regular_Hours Overtime_Hours Total_Hours
0  Arun           8.0           2.0         10.0
2  Charan          9.0           3.0         12.0
>>>

```

PROBLEM CODING

```

import pandas as pd
n = int(input("Enter number of equipment: "))
data = []
for i in range(n):
    name = input("Enter equipment name: ")
    voltage = float(input("Enter voltage: "))
    current = float(input("Enter current: "))
    pf = float(input("Enter power factor: "))
    data.append([name, voltage, current, pf])
df = pd.DataFrame(
    data,
    columns=["Equipment", "Voltage", "Current", "Power_Factor"]
)
df["Apparent_Power"] = df.eval("Voltage * Current / 1000")
df["Useful_Power"] = df.eval("Apparent_Power * Power_Factor")
limit = float(input("Enter power factor limit: "))
result = df.query("Power_Factor < @limit")
print("\nEquipment below the limit:")
print(result)

```

OUTCOMES OF PROGRAM CODING

```

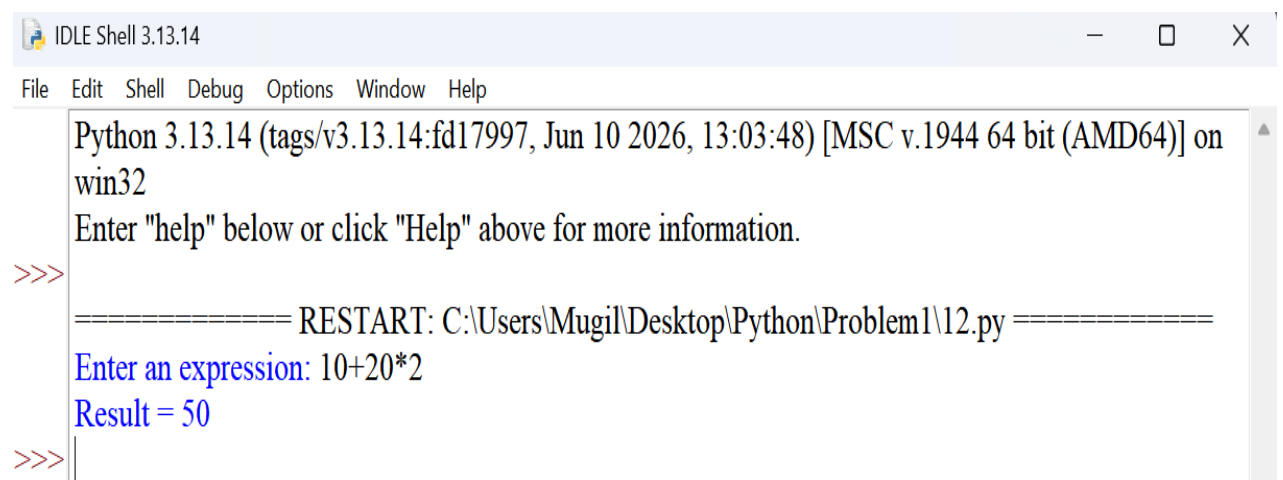
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem\experiment12.py =====
Enter number of equipment: 2
Enter equipment name: Motor
Enter voltage: 230
Enter current: 10
Enter power factor: 0.8
Enter equipment name: Transformer
Enter voltage: 230
Enter current: 5
Enter power factor: 0.95
Enter power factor limit: 0.9

Equipment below the limit:
Equipment Voltage Current Power_Factor Apparent_Power Useful_Power
0 Motor 230.0 10.0 0.8 2.3 1.84

```

PROBLEM CODING

```
var = input("Enter an expression: ")  
result = eval(var)  
print("Result =", result)
```

OUTCOMES OF PROGRAM CODING

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>> ===== RESTART: C:\Users\Mugil\Desktop\Python\Problem1\12.py =====  
Enter an expression: 10+20*2  
Result = 50  
>>> |
```

VIVA QUESTION**1.What is the purpose of the eval() method in Pandas?**

The eval() method in Pandas is used to evaluate a string expression in the context of a DataFrame. It allows for faster operations on DataFrame columns by enabling the use of expressions involving column names directly, rather than requiring the use of standard arithmetic operators on columns separately.

2.How does the query() method differ from the eval() method?

The query() method is specifically designed for filtering DataFrames based on a query expression, while the eval() method evaluates expressions and can be used to create new columns or perform calculations. query() is more intuitive for filtering data, whereas eval() focuses on evaluating arithmetic or logical expressions involving columns.

3. Can you provide an example of how to use the eval() method to create a new column in a DataFrame?

Yes! For example, if you have a DataFrame with columns Salary and Bonus, you can create a new column Salary_After_Bonus by evaluating the expression directly:

```
df['Salary_After_Bonus'] = df.eval('Salary + Bonus')
```

This will add the Bonus to Salary and store the result in the new column.

4. What are the benefits of using the query() method for filtering DataFrames?

The benefits of using the query() method include:

- **Readability:** It allows for a more intuitive and readable syntax for filtering DataFrames, resembling SQL queries.
- **Performance:** It can be faster than traditional filtering methods, especially on large DataFrames, because it uses an expression language that can optimize the evaluation.
- **Convenience:** You can refer to columns directly without needing to use the DataFrame's bracket notation.

5.Is it possible to use variables in query() expressions? If so, how?

Yes, you can use variables in query() expressions by prefixing the variable names with the @ symbol. For example:

```
threshold = 50000
```

```
result = df.query('Salary > @threshold')
```

In this example, @threshold refers to the variable threshold, allowing the query() method to filter the DataFrame based on its value.

RESULT:

Thus, the Python program successfully demonstrated the use of `eval()` and `query()` functions for evaluating expressions and filtering data from a DataFrame. The required conditions were executed correctly, and the results were displayed and verified successfully.

INFERENCE:

The experiment demonstrates that `eval()` can efficiently evaluate expressions involving DataFrame columns, while `query()` provides a convenient way to filter records based on specified conditions. These functions simplify data manipulation, analysis, and conditional data selection in Python using Pandas.

Assessment	Max. Marks	Marks Obtained
Conduction of Practical	15	
Report	5	
Viva	5	
Total Marks	25	
Signature of Lab In-Charge		